

PROJECT AND TEAM INFORMATION

Project Title

(Try to choose a catchy title. Max 20 words).

Smart Playlist Shuffler: A Non-Repeating Audio Engine

Student / Team Information

Team Name: Team #	DSCPP-III-2026-T142
Team member 1 (Team Lead)	Bhuvi Singhal -- 2510010271 2510010271@geu.ac.in 8218385053 
Team member 2	Akshat Joshi -- 2510012740 akshat2joshi06@gmail.com 9258812302 
Team member 3	Akshank Chauhan -- 2510010897 2510010897@geu.ac.in 7017935701 
Team member 4	Tansi Singh -- 2510010500 2510010500@geu.ac.in 9696951997 

PROPOSAL DESCRIPTION (10 pts)

Motivation (1 pt)

(Describe the problem you want to solve and why it is important. Max 300 words).

It's pretty annoying when you put a carefully curated playlist on shuffle and end up hearing the exact same track again just a few songs later. Basic media players do this all the time, and it completely ruins the listening experience. The underlying issue is that they usually rely on a basic random math function—something like a standard `rand()` call in C++. That function just picks a random index number out of thin air every single time the current song ends. It has absolutely zero memory of what actually played two minutes ago. Because of how basic probability works, true randomness almost always creates track clustering. This means a handful of songs will play constantly, while others sit at the bottom of the list and get completely ignored before the cycle restarts.

We decided to fix this by building a smart shuffler that actively tracks playback history. Our core logic guarantees that you absolutely won't hear a repeat until every single song in the file has had its turn.

We picked this specific idea for our PBL project because it gives us a highly practical reason to build custom data structures from scratch. A lot of typical beginner projects, like library or student management systems, are mostly just basic data entry. You add an item to a list, and you delete an item. We didn't want to do that. Building this shuffler forces us to tackle a real algorithmic logic problem. We have to manually map out raw memory pointers, handle state retention across a continuous loop, and actively prevent data collisions using hash sets. It is a much more hands on way to see exactly how shuffle algorithms and circular linked lists behave in a real, functioning piece of software, rather than just reading about them in a textbook.

State of the Art / Current solution (1 pt)

(Describe how the problem is solved today (if it is). Max 200 words).

If you look at most beginner coding projects or simple command line media players, they usually just call a basic pseudo random number generator to pick the next track. The code essentially asks for a random integer up to the total playlist size and plays whatever index comes back. Since these programs don't look back at what just happened, they have zero memory of the playback history. Because of how basic probability works, tracks naturally start colliding and repeating way too early.

Big companies like Spotify actually ran into this exact issue years ago. Their users kept complaining that the shuffle didn't feel random at all. They ended up solving it by writing complex algorithms that actively space out artists and genres. We definitely aren't trying to build a massive framework like that. We just want to take that core foundational idea strictly stopping early repetition and build it completely from scratch using raw C++. Doing this manually gives us a much stronger grasp on how data structures actually govern playback sequences.

Project Goals and Milestones (2 pts)

(Describe the project general goals. Include initial milestones as well any other milestones. Max 300 words).

Our main objective is building a terminal based audio engine that prevents track repetition during a single playback cycle. We have to write code for the Fisher-Yates algorithm and all core data structures by ourselves. Since the Standard Template Library (STL) is not allowed, engineering custom node classes and handling raw memory allocation forms a major part of our technical workload. We split the development into three specific milestones. The first is architectural planning. We spent time mapping out exactly how the pointers in our Circular Doubly Linked List (CDLL) will connect. Keeping an endless loop running without causing memory crashes means we have to map things carefully. You can't just push data into a static array if you want a playlist to loop endlessly without crashing. Phase two is the actual C++ implementation.

We need to program a custom hash set for $O(1)$ tracking of executed song IDs, alongside a LIFO stack to manage the skip undo state. Calibrating the Fisher-Yates swap logic to confirm an unbiased randomized queue is another critical step. Systematically resolving hash collisions will likely be the hardest logic problem here, since we have to write the probing sequence manually. The third milestone focuses on system integration and stability.

We will link the application with the Windows Multimedia API to process .wav files asynchronously. A huge portion of this phase involves memory profiling. We have to check that our custom class destructors properly release all heap allocations during termination, ensuring no memory leaks occur before our final evaluation.

Project Approach (3 pts)

(Describe how you plan to articulate and design a solution. Including platforms and technologies that you will use. Max 300 words).

We are writing the entire project in standard C++. Since STL containers are strictly off the table, we have to build every single data structure from scratch. That puts the burden of class architecture, raw pointer traversal, and dynamic memory allocation entirely on us. For compilation, we'll just use GCC or MSVC.

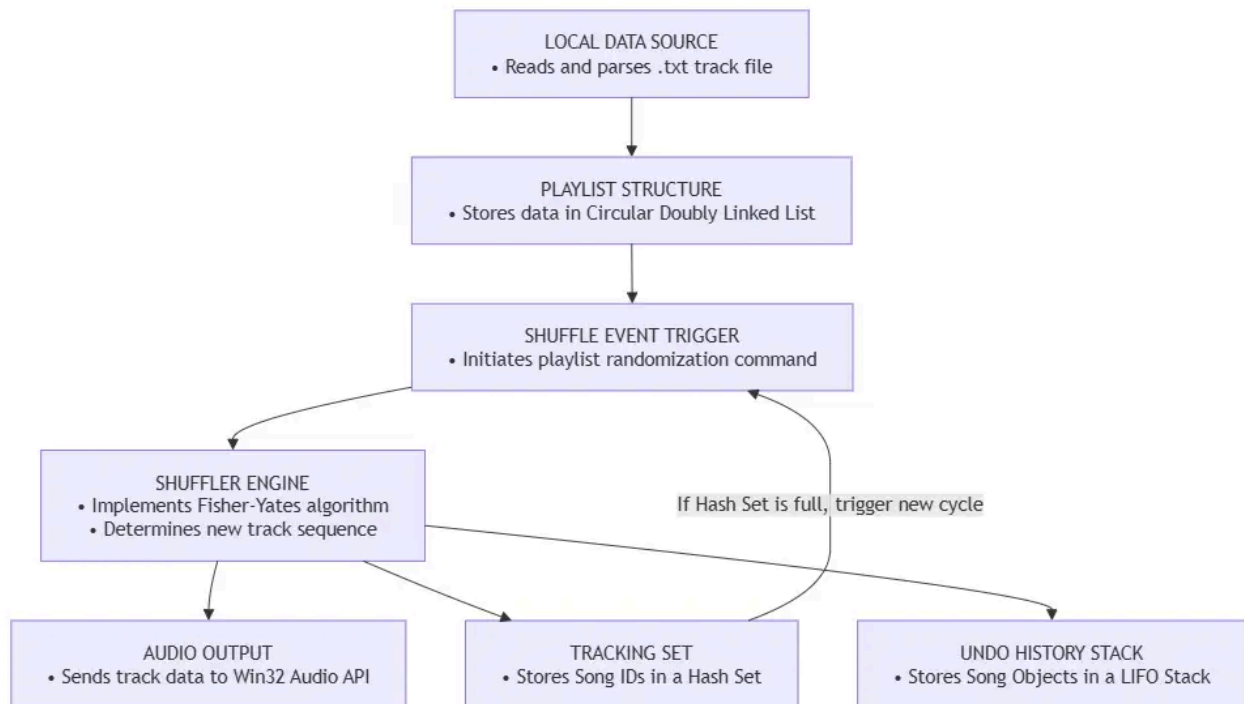
The playlist itself lives inside a custom Circular Doubly Linked List. Playlists loop naturally anyway. When you hit the last track, moving forward just wraps the pointer right back to the start. Instead of hardcoding the songs, the app pulls track IDs and local file paths directly from a basic text file.

When a user hits play, the linked list copies into a temporary array so we can run a Fisher-Yates shuffle. This guarantees a mathematically fair queue in $O(n)$ time. As songs finish, their unique IDs drop into our hash set. Before the player loads the next track, it checks that set. If the set is full, the program wipes the tracking data and reshuffles everything. We also threw in a LIFO stack. That way, if a user wants to undo a skip, we just pop the stack to get the exact previous track.

For the audio output, we went with the Win32 API and mmsystem.h. Passing the SND_ASYNC flag is critical here because it pushes the audio execution to a background thread. If we didn't use that flag, the audio buffer would lock up the main thread and completely freeze our terminal interface.

System Architecture (High Level Diagram)(2 pts)

(Provide an overview of the system, identifying its main components and interfaces in the form of a diagram using a tool of your choice).



Project Outcome / Deliverables (1 pts)

(Describe what are the outcomes / deliverables of the project. Max 200 words).

By the end of the semester, we are going to hand in a fully functional terminal based audio player. It will actually read real .wav files from the hard drive and play them. Our main goal here is to physically prove that our zero repeat shuffle logic works exactly as we planned. The most important thing we're delivering is the raw C++ code itself. That code is the whole point of the project because it shows exactly how we managed to link all these custom data structures together to solve a very specific algorithmic problem without relying on any standard libraries.

Along with the source files, we will also provide the compiled .exe file so anyone can test it directly on their computer, plus the playlist.txt file that acts as our database. Finally, we are going to include a short memory test log to show that our destructors work correctly.

Assumptions

(Describe the assumptions (if any) you are making to solve the problem. Max 100 words)

We're assuming the user has local .wav files saved right in the same folder as the executable. We also assume the playlist won't be massive, probably just a few hundred songs at most so our array exports don't run into memory limits. Since we use the Win32 API for sound, this program assumes it's running on a Windows machine.

References

(Provide a list of resources or references you utilised for the completion of this deliverable. You may provide links).

- 1. D.E. Knuth's The Art of Computer Programming : We used his specific breakdown to write the Fisher-Yates shuffle array.*
- 2. Microsoft MSDN documentation: We had to dig through the mmsystem.h pages to figure out how to stop the audio playback from freezing our terminal interface.*
- 3. Spotify Engineering Post : "How to shuffle songs." It does a really good job explaining why basic random functions are terrible for music playlists.*
- 4. GeeksforGeeks and Cppreference: We referenced these heavily just to make sure we weren't messing up the custom node pointers and memory deallocation.*